Cupcake Projekt 2. semester

Projekt Start: 08-Apr-2026

Projekt Slut:

Rapport Start: 08-Apr-2026

Rapport Slut:

Studerende:

Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Indholdsfortegnelse:

Indhold

Indholdsfortegnelse: 1

Indledning: 2

Krav: 3

Aktivitetsdiagram: 4

Domæne model og ER diagram: 5

Navigationsdiagram: 9

Særlige forhold: 10

Status på implementation: 13

Proces: 14

Studerende:
Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Indledning:

Denne rapport er udarbejdet med henblik på at dokumentere og forklare opbygningen af vores projekt. Rapporten er skrevet til en fagfælle, dvs. en datamatikerstuderende på 2. semester, som ikke på forhånd kender til cupcake-projektet. Formålet er at give læseren tilstrækkelig indsigt til at kunne forstå projektets opbygning, teknologivalg og implementering.

Projektet tager udgangspunkt i den viden og de kompetencer, vi har opnået gennem 2. semester på datamatikeruddannelsen. Opgaven består i at udvikle en fullstack webapplikation baseret på en kundes behov og forretningskrav, som er blevet præsenteret i form af user stories (US).

❖

Baggrund:

Projektet omhandler udviklingen af et system til en cupcakebutik på Bornholm. Virksomheden ønsker at digitalisere deres bestillingsproces, så kunder kan foretage ordrer online i stedet for fysisk i butikken.

Systemet skal gøre det muligt for kunder at oprette en profil og bestille cupcakes ved selv at sammensætte deres produkt. Dette indebærer, at kunden kan vælge både bund og topping. Derudover skal kunden kunne gemme eller slette en ordre, se sine bestillinger, betale for og afhente sin ordre på et senere tidspunkt i selve butikken.

Ud over kundens funktionalitet skal systemet også understøtte en administrativ rolle. En administrator skal have mulighed for at håndtere ordrer, herunder afbryde dem, samt administrere brugernes økonomi, eksempelvis ved at flytte penge mellem profiler.

Begge skal kunne se deres log ind hele tiden i en øvre bjælke.

❖

Teknologivalg:

Projektet er udviklet som en fullstack webapplikation med fokus på at anvende de teknologier og principper, der er introduceret på uddannelsens 2. semester.

En mock-up af webapplikationen er lavet i Figma (126.2.10).

Applikationen er implementeret i Java (Java Amazon Coretto 17.0.18) og udviklet i IntelliJ (2025.2.2) med Maven(4.0.0). Weblaget er opbygget ved brug af Javalin(6.7.0) som framework samt Thymeleaf(3.1.3.RELEASE) som template engine til generering af dynamiske HTML-sider. Frontend-delen består af HTML, CSS og JavaScript.

Databasen er implementeret i PostgreSQL (16.2) og køres via Docker (4.59.0) Forbindelsen mellem applikation og database håndteres gennem JDBC. Databasen er designet efter principperne for 3. normalform for at sikre en struktureret og konsistent datamodel.

Studerende:

Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Krav:

Kunden håber, at denne webapplikation kan skabe en brugervenlig oplevelse, som gør det attraktivt for kunderne at bestille deres cupcakes online. Systemet skal sikre, at det opleves som værdifuldt for kunden at anvende applikationen frem for at bestille fysisk i butikken.

Derudover er det et krav, at alle funktionelle behov bliver opfyldt, så kunderne har mulighed for at sammensætte deres cupcakes præcis efter egne ønsker.

Det færdige produkt skal kunne bidrage til at tiltrække flere kunder og øge omsætningen. Samtidig skal systemet understøtte en enkel og professionel administration, så det er nemt for administratoren at håndtere ordrer og kundedata.

❖

User stories:

US-1: Som kunde kan jeg bestille og betale cupcakes med en valgfri bund og top, sådan at jeg senere kan køre forbi butikken i Olsker og hente min ordre.

US-2 Som kunde kan jeg oprette en konto/profil for at kunne betale og gemme en en ordre.

US-3: Som administrator kan jeg indsætte beløb på en kundes konto direkte i Postgres, så en kunde kan betale for sine ordrer.

US-4: Som kunde kan jeg se mine valgte ordrelinier i en indkøbskurv, så jeg kan se den samlede pris.

US-5: Som kunde eller administrator kan jeg logge på systemet med email og kodeord. Når jeg er logget på, skal jeg kunne se min email på hver side (evt. i topmenuen, som vist på mockup'en).

US-6: Som administrator kan jeg se alle ordrer i systemet, så jeg kan se hvad der er blevet bestilt.

US-7: Som administrator kan jeg se alle kunder i systemet og deres ordrer, sådan at jeg kan følge op på ordrer og holde styr på mine kunder.

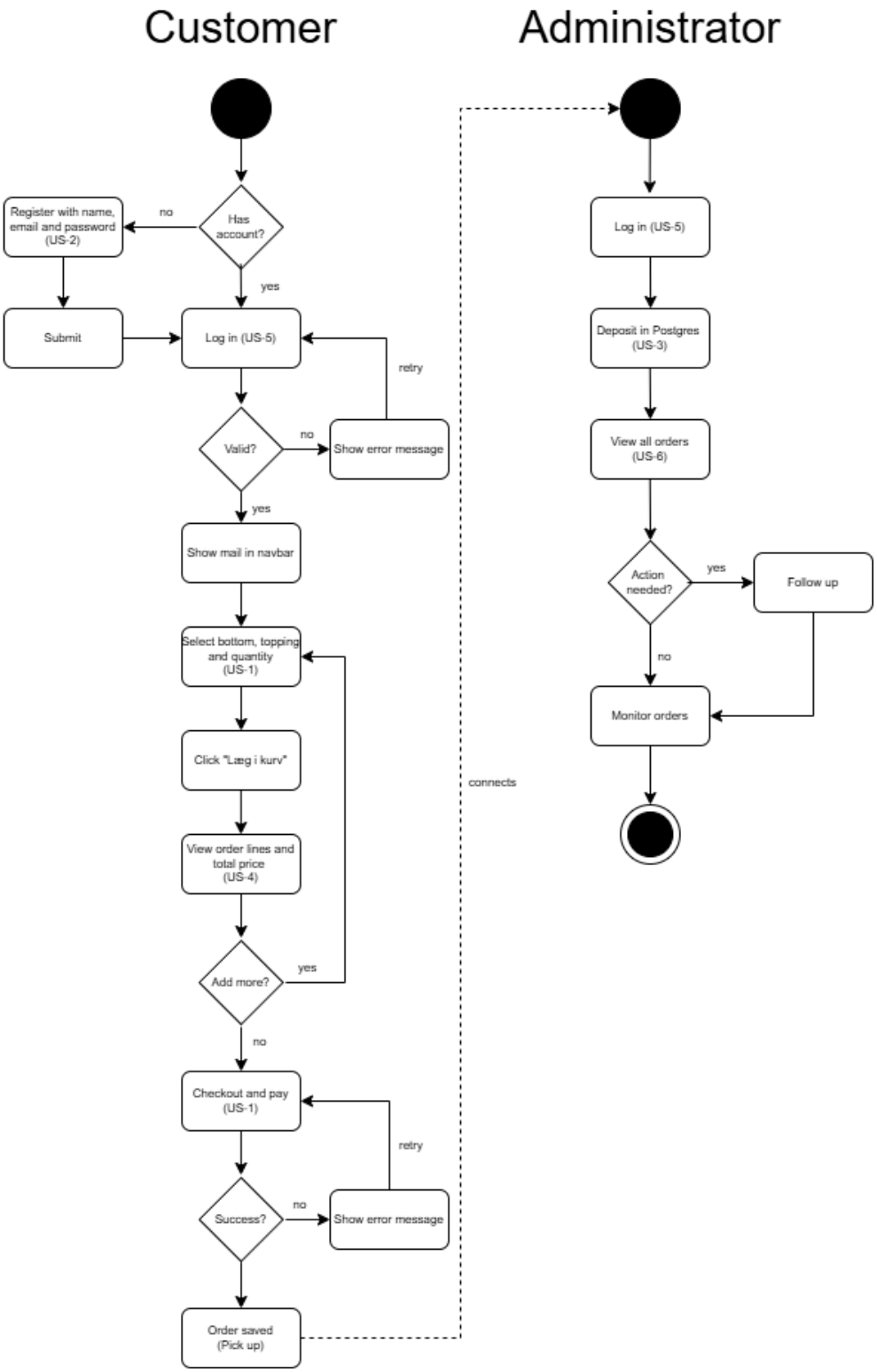
US-8: Som kunde kan jeg fjerne en ordrelinie fra min indkøbskurv, så jeg kan justere min ordre.

US-9: Som administrator kan jeg fjerne en ordre, så systemet ikke kommer til at indeholde udgyldige ordrer. F.eks. hvis kunden aldrig har betalt.

Studerende:
Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Aktivitetsdiagram:

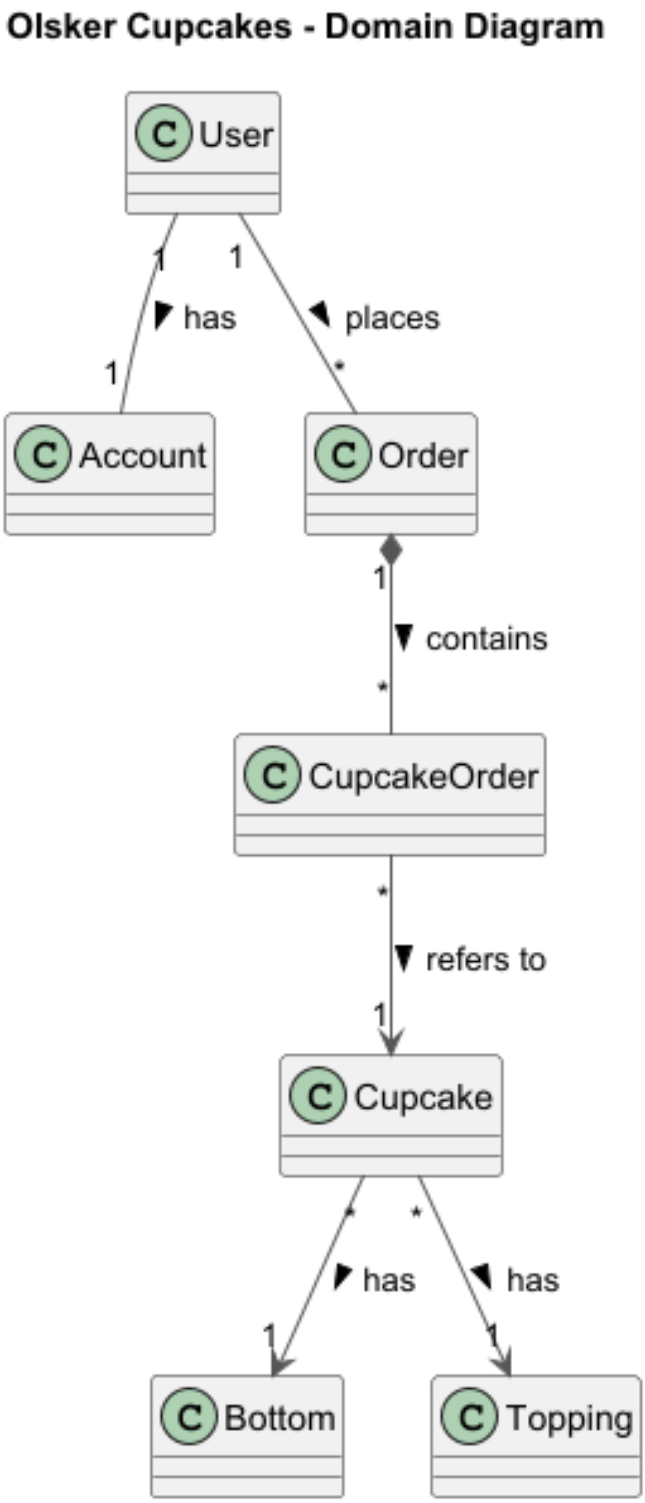


Studerende:
Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Domæne model og ER diagram:

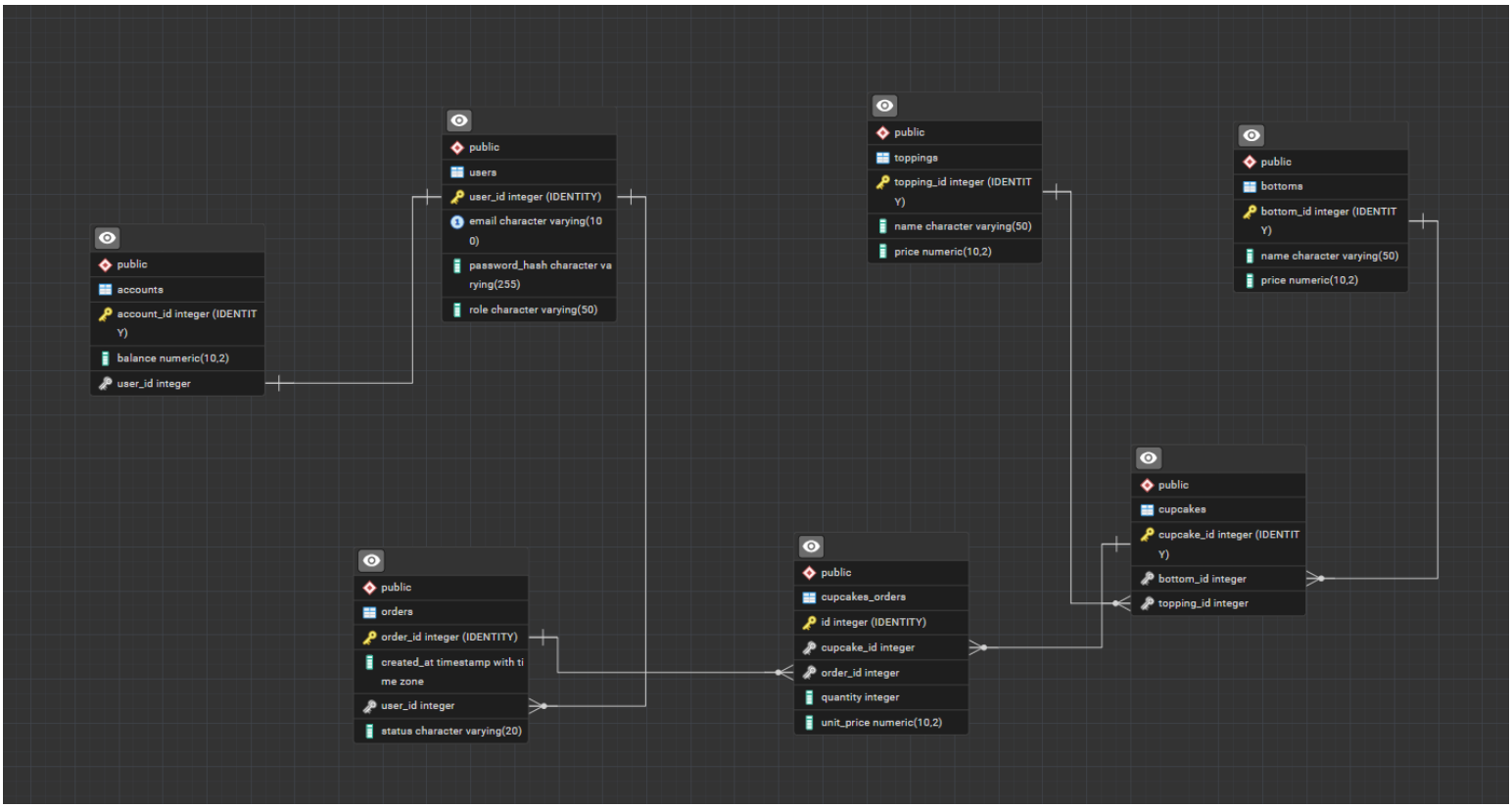
Domænemodel:



Studerende:
Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

ER diagram:



ERD-model – overblik og designvalg:

Databasen er lavet til et cupcakesystem, hvor brugere kan bestille cupcakes. Hver cupcake består af en bund og en topping, og ordrer kan indeholde flere cupcakes.

❖

Tabeller i modellen:

Modellen består af følgende tabeller: users, accounts, orders, cupcakes, cupcakes_orders, toppings og bottoms. Hver tabel repræsenterer en tydelig del af systemet og er adskilt for at undgå gentagelse af data og gøre strukturen mere fleksibel.

❖

Relationer i databasen:

Users og accounts

Der er en relation mellem users og accounts, hvor accounts.user_id peger på users.user_id. Det er en 1-til-1 relation.

Studerende:
Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Grunden til at de er delt i to tabeller er, at login-oplysninger (som password og rolle) holdes adskilt fra brugerdata. Det giver bedre sikkerhed og gør det nemmere at potentielt udvide systemet senere.

Users og orders

Der er en 1-til-mange relation mellem users og orders, fordi en bruger kan have flere ordrer, mens hver ordre kun tilhører én bruger. Det styres via orders.user_id.

Orders og cupcakes

Der er en mange-til-mange relation mellem orders og cupcakes, som er løst med tabellen cupcakes_orders.

I denne tabel gemmes både hvilke cupcakes der er i ordren, hvor mange der er bestilt, og prisen pr. styk. Det sidste er vigtigt for at sikre, at man kan se den historiske pris, selv hvis priserne ændrer sig senere.

❖

Cupcakes, toppings og bottoms:

En cupcake består af én bund og én topping.

- cupcakes.bottom_id peger på bottoms
- cupcakes.topping_id peger på toppings

Det betyder, at mange cupcakes kan bruge de samme toppings og bunde, men hver cupcake har kun én af hver.

❖

Normalisering:

Modellen er overordnet set normaliseret til 3. normalform, fordi data er opdelt i separate tabeller uden unødvendige gentagelser, og alle ikke-key kolonner er funktionelt dependent af hele primær nøgle constrainten for tabellen.

Der er dog et bevidst valg i cupcakes_orders, hvor der gemmes enhedspris. Det er gjort for at sikre, at man kan genskabe den rigtige pris på en ordre, selv hvis priserne ændrer sig senere. Det er altså et praktisk valg frem for et rent teoretisk normaliseringskrav.

❖

Studerende:

Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Nøgler og struktur:

Alle tabeller bruger et automatisk genereret ID som primærnøgle/Surrogatnøgle. Det gør databasen mere stabil, fordi relationerne ikke afhænger af data som navne eller andre værdier, der kan ændre sig.

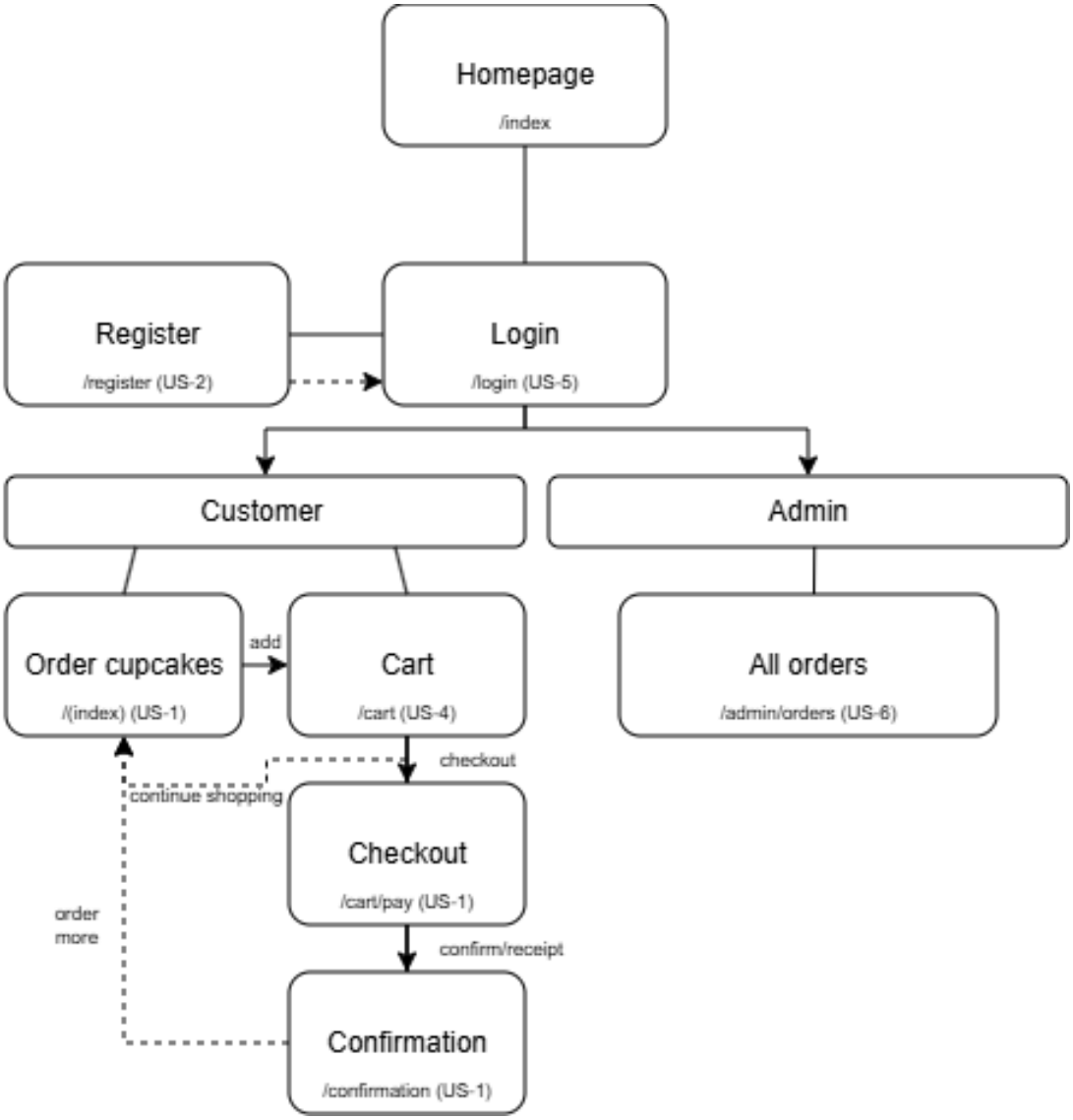
Mange-til-mange relation:

Den eneste mange-til-mange relation er mellem orders og cupcakes, som er løst korrekt med koblingstabellen cupcakes_orders. Det er en standard løsning i relationelle databaser.

Studerende:
Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Navigationsdiagram:



Studerende:
Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Særlige forhold:

Vi gemmer i sessions for denne løsning:

session attribute: currentUser

Denne indeholder et User-objekt med:

- userId
- email
- role

Formål:

- Bruges til at sikre login-status
- Bruges til at hente brugerens ordrer og data
- Bruges til at vise brugerens email i UI (US-5)

❖

Vi håndterer exceptions i controller-laget med try/catch:

Service layer throws DatabaseException.

Controller catches exception and returns HTTP error response.

Formål:

- Undgå server crash
- Give brugeren en fejlbesked
- Sikre stabilitet i webapplikationen

Eksempel:

```
try {
   ctx.json(CartService.getCart(orderId, cp));
} catch (DatabaseException e) {
   ctx.status(500).result("Error fetching cart: " + e.getMessage());
}
```

❖

Studerende:
Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Validering sker primært i controller-laget.

Vi kontrollerer:

- om input er null
- om tal kan parses korrekt
- om bruger er logget ind

Eksempel:

```
int orderId = Integer.parseInt(ctx.pathParam("orderId"));
```

Hvis input er ugyldigt, håndteres det via exception handling.

❖

Login sikkerhed håndteres via AuthService.

- Brugeren findes via email
- Password sammenlignes med databasen
- Ved succes gemmes bruger i session

Eksempel:

```
ctx.sessionAttribute("currentUser", user);
```

Sikkerhed:

- Kun autentificerede brugere kan tilgå ordrer
- Session bruges til at validere adgang

❖

Brugertyper i databasen findes feltet:

role VARCHAR(50)

Typiske roller:

- customer
- admin

Studerende:
Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Brug i JDBC:

rs.getString("role")

Formål:

- Bestemme adgangsniveau
- Admin kan se alle ordrer (US-6)
- Customer kan kun se egne ordrer

❖

Lagdelt arkitektur

- Controller → HTTP håndtering
- Service → business logic
- Mapper → database adgang
- Entity → data model

❖

Bestilling af cupcakes

- Cupcakes består af bottom + topping
- Mange-til-mange relation via cupcakes_orders

❖

Prisberegning

- Total beregnes i SQL med SUM(quantity * unit_price)
- Undgår beregningsfejl i Java

Studerende:

Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Status på implementation:

Da vi kom langsomt fra start tog vi den bevidste beslutning at kun at gå efter minimumskravet af implementation af User Stories 1-6 og springe 7-9 over (se krav).

Nogle af de ting vi ikke har kunnet nå er disse:

- Betalingsflow er ikke implementeret, man kan som kunde bastille cupcakes med valgfri top og bund, men betalingen mangler.

- Man kan som administrator blot se kunders ordrer, men ikke indsætte beløb på en kundens konto i Postgres.

- Endpoint for betaling mangler grundet manglende implementation.

- Endpoint for ”Om os” siden er ikke implementeret, men er blot til stede for æstetik.

- Kurv-ikonet oppe i højre hjørne bør tage brugeren til checkout-flowet, men dens endpoint kan ikke findes. Dette var en last-minute opdagelse og vi havde ikke tid til at rette det.

Vi håber at kunne nå alt vi gerne vil til næste projekt.

Studerende:

Elvira Frederikke Savia Birtø cph-eb120@stud.ek.dk
Github: Eligoon Klasse: 0925B

Kojo Essel Ramsby cph-kr281@stud.ek.dk
Github: LoveBytes25 Klasse: 0925A

Proces:

Hvad var jeres planer for teamets arbejdsform og projektforløbet?

Vi havde ikke lagt nogle konkrete planer fra starten, da vi tidligt i forløbet blev ramt af sygdom i gruppen. Vi havde en fin kommunikation undervejs, men det gjorde det svært at få fordelt roller og ansvar tydeligt mellem os.

❖

Hvordan kom det til at forløbe i praksis?

I praksis havde vi en langsom start, hvor der ikke skete så meget i begyndelsen. Derfor valgte vi at fokusere på at få lavet en struktur og et skelet til rapporten, så vi kunne arbejde hurtigere mod slutningen, hvor vi forventede at blive tidspressede.

❖

Hvad gik godt, og hvad kunne have været bedre?

Vi arbejdede godt sammen som team, især når vi var under pres, og vi var hurtige til at sætte os ind i nye metoder og kode.
Til gengæld kunne vi have haft gavn af at komme tidligere i gang, men det blev gjort sværere af sygdom i gruppen.

❖

Hvad har I lært af processen, og hvad vil I gøre anderledes næste gang?

Vi har lært, at det er vigtigt at arbejde mere struktureret og i en klar rækkefølge. Næste gang vil vi være tydeligere omkring, hvem der laver hvad, og hvorfor det er vigtigt. Derudover vil vi forsøge at fordele arbejdet mere jævnt gennem hele forløbet.